Analysis Report 2

Grant Spivey

2025-04-08

```
movies <- read.csv("C:/Users/mandu/Downloads/archive/IMDB Top 250 Movies.csv")
```

Data Description

The data I am using for this report is titled "IMDB Top 250 Movies Dataset" and comes from kaggle.com. The dataset contains data about the top 250 highest rated movies from the Internet Movie Database (IMDB) in 2021. The dataset contains information on the movies including their name, ranking, release year, rating, genre, budget, box office, run-time, certificate (parental guidance rating) and more.

Research Question

The research question guiding the report is: "What are the best predictors for IMDB ratings among the 250 movies?" I think it will be interesting to sift through the data and see if there are patterns that differentiate the movies that are deemed the best of the best and whether certain qualities of the movie may predict that movie's critical success.

Variables of Interest

The variables I will explore to build my predictive model are:

Dependent (what I am predicting) 1. rating - this variable is numerical and is the IMDB rating of the movie and the dependent variable which I am trying to predict.

Independent (factors used to make a prediction) 2. year - this variable is numerical and is the year of release of the movie

3. run_time - this variable is supposed to be numerical for purposes of my model but is stored as a string in "h m" format; the variable contains the run time of the movie
4. budget - this variable is supposed to be numerical but is stored as a string and is the budget of the movie
5. box_office - this variable is supposed to be numerical but is stored as a string and is the box office of the movie (gross income)

Data Wrangling

The main data wrangling that needs to be done is to change the string variables into numeric integers so I can make my model without error. I also want to remove movies that have some missing data.

```
#Converting run_time to numerical minutes
require(tidyverse)
```

```
## Loading required package: tidyverse

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(stringr)

movies$run_time <- sapply(movies$run_time, function(x) {
  hours <- as.numeric(str_extract(x, "\\d+(?=h)"))
  minutes <- as.numeric(str_extract(x, "\\d+(?=m)"))

  if (is.na(hours)) hours <- 0
  if (is.na(minutes)) minutes <- 0

  return(hours * 60 + minutes)
})

head(movies$run_time)
```

```
## [1] 142 175 152 202  96 195
```

```r
#Now to tackle budget and convert it to just a numerical variable

movies$budget <- as.numeric(gsub("[^0-9]", "", movies$budget))

movies$budget[is.na(movies$budget)] <- NA

head(movies$budget)
```

```
## [1] 2.50e+07 6.00e+06 1.85e+08 1.30e+07 3.50e+05 2.20e+07
```

```r
#Lastly, the box office transformed into a numerical variable

movies$box_office <- gsub("[^0-9]", "", movies$box_office)

movies$box_office <- as.numeric(movies$box_office)

movies$box_office[is.na(movies$box_office)] <- NA

head(movies$box_office)
```

```
## [1]   28884504  250341816 1006234167   47961919        955  322161245
```

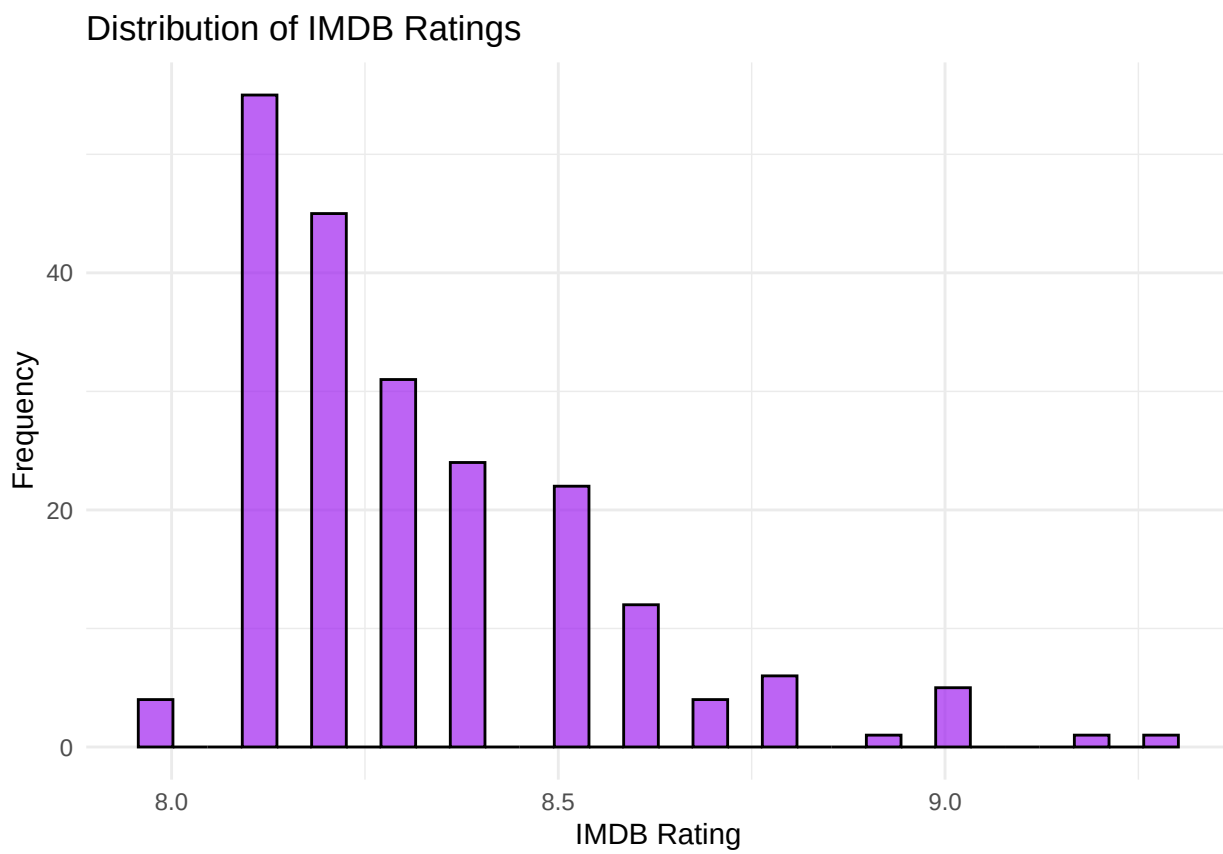
```r
#Now to clean the dataset by removing movies that are missing some values

movies_clean <- movies %>%
  filter(!is.na(box_office) & !is.na(budget))
```

Exploratory Data Analysis

There are lots of visualizations to look at to understand trends within the data.

Ratings

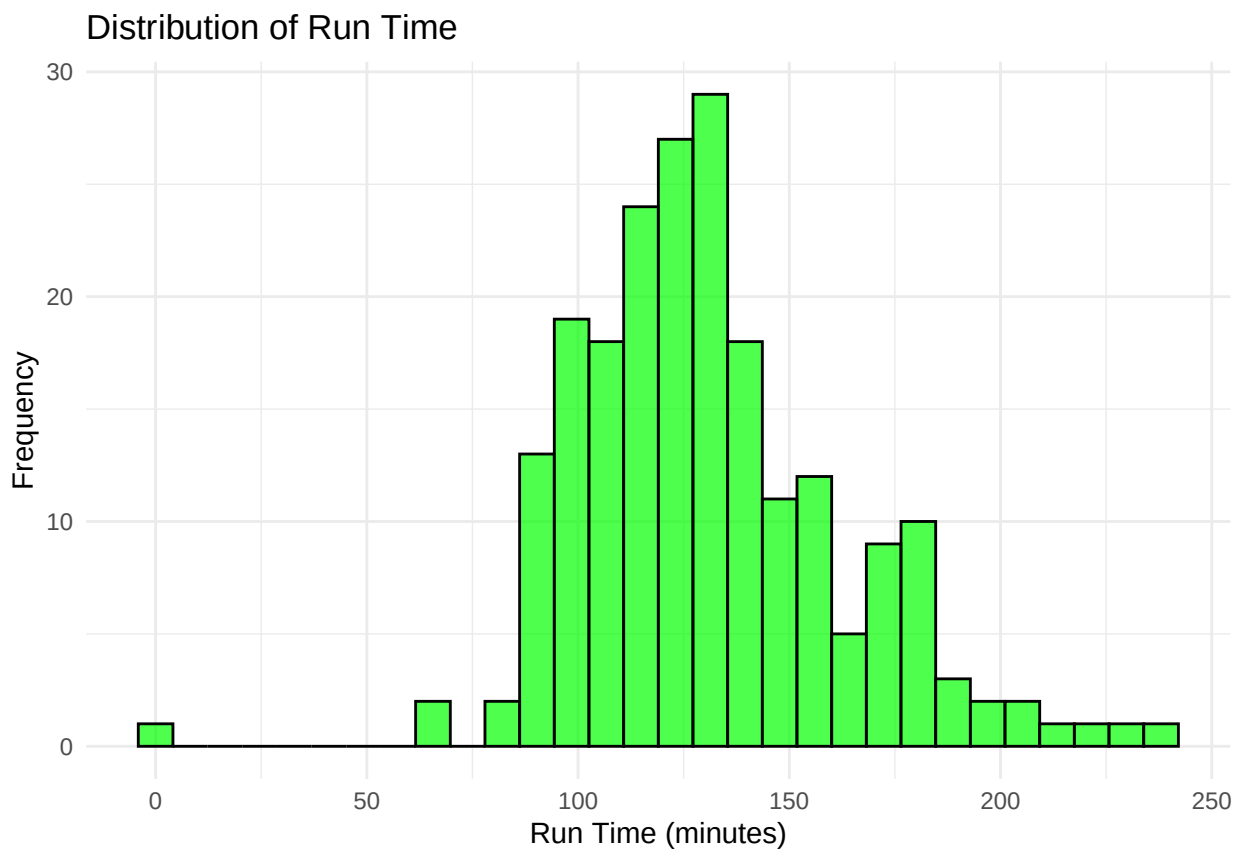
```
ggplot(movies_clean, aes(x = rating)) +
  geom_histogram(bins = 30, fill = "purple", color = "black", alpha = 0.7) +
  labs(title = "Distribution of IMDB Ratings", x = "IMDB Rating", y = "Frequency") +
  theme_minimal()
```



It seems that the ratings are right skewed; most of the movies on the list of the 250 best rated movies have ratings between 8 and 8.5.

Run Time

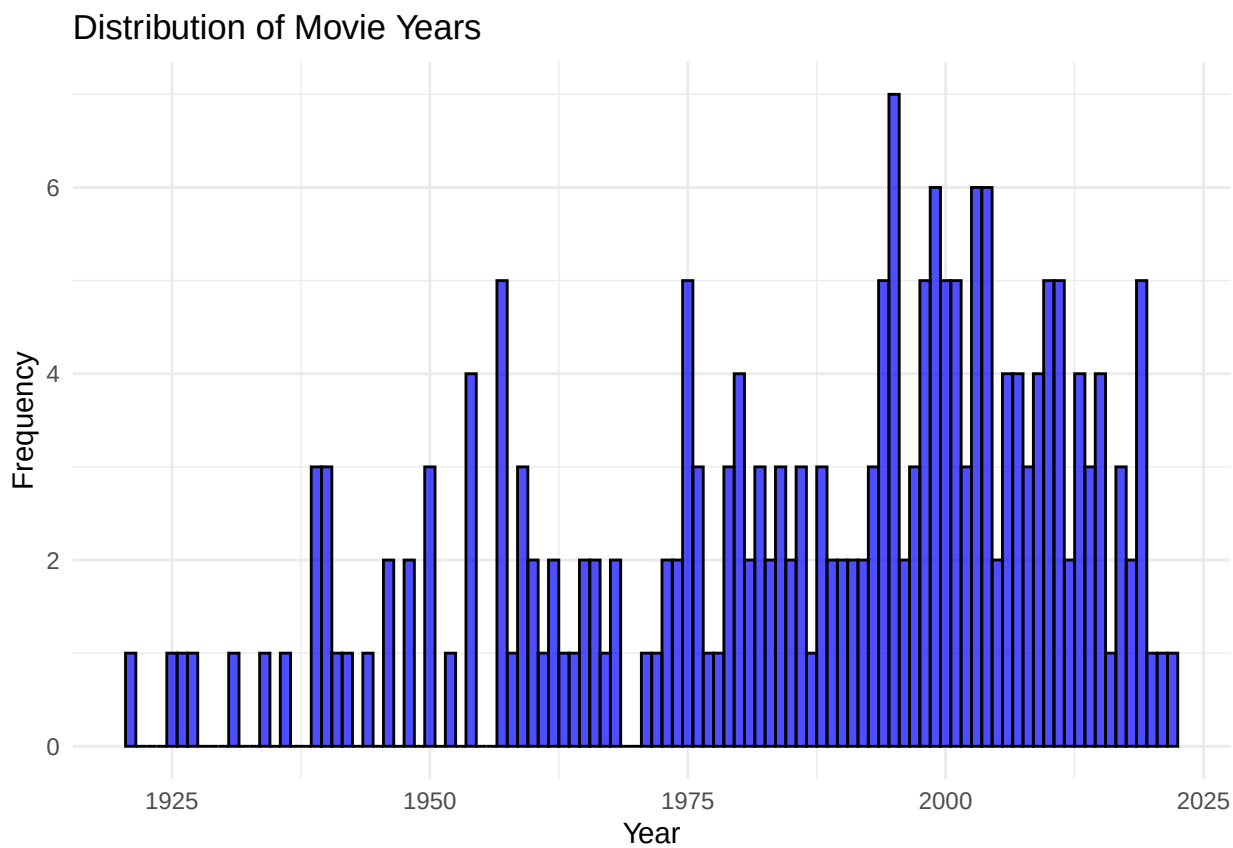
```
ggplot(movies_clean, aes(x = run_time)) +
  geom_histogram(bins = 30, fill = "green", color = "black", alpha = 0.7) +
  labs(title = "Distribution of Run Time", x = "Run Time (minutes)", y = "Frequency") +
  theme_minimal()
```



We see that the run time is actually quite normally distributed, with the majority of movies falling between 100 and 150 minutes in run time.

Year

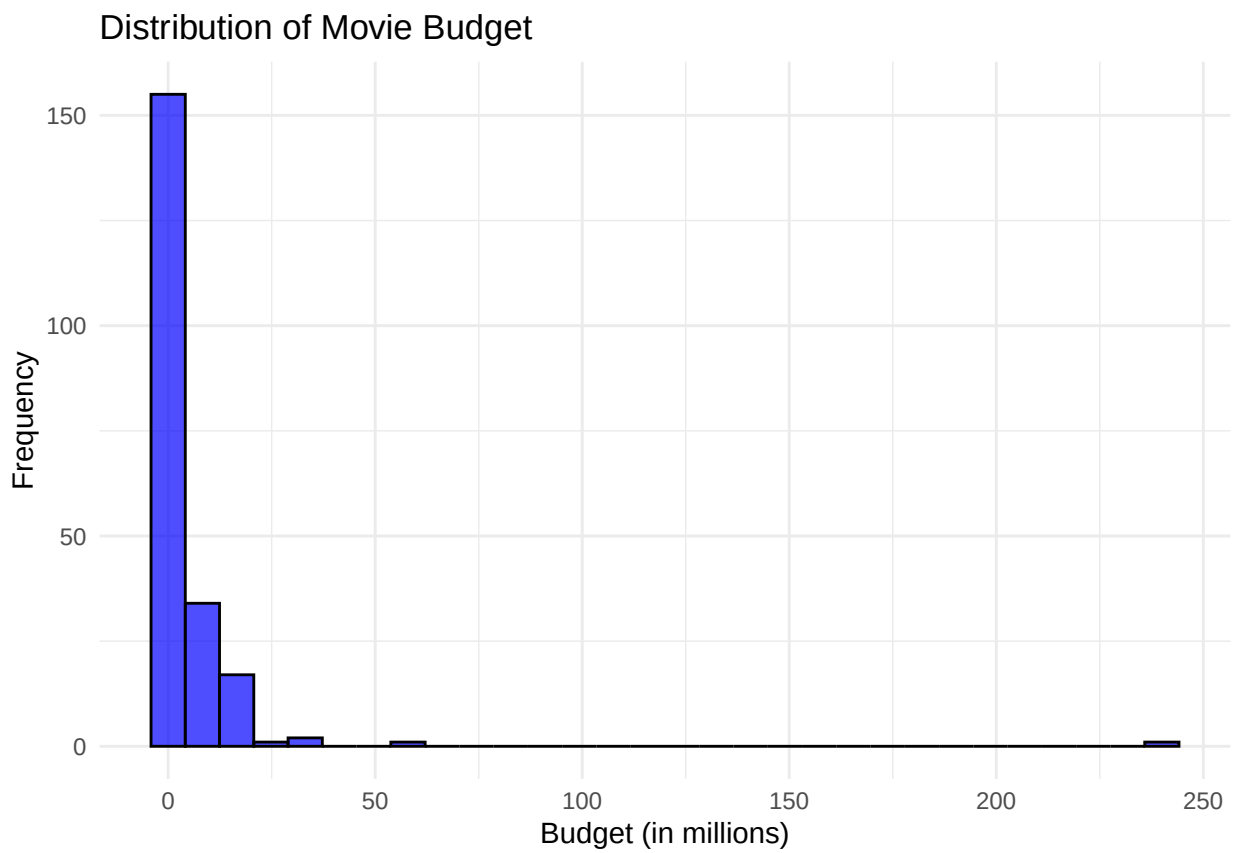
```
ggplot(movies_clean, aes(x = year)) +
  geom_histogram(binwidth = 1, fill = "blue", color = "black", alpha = 0.7) +
  labs(title = "Distribution of Movie Years", x = "Year", y = "Frequency") +
  theme_minimal()
```



Based on this visualization, we see that more of the top rated movies came out more recently, between the years of the late 90s to the mid 2010s.

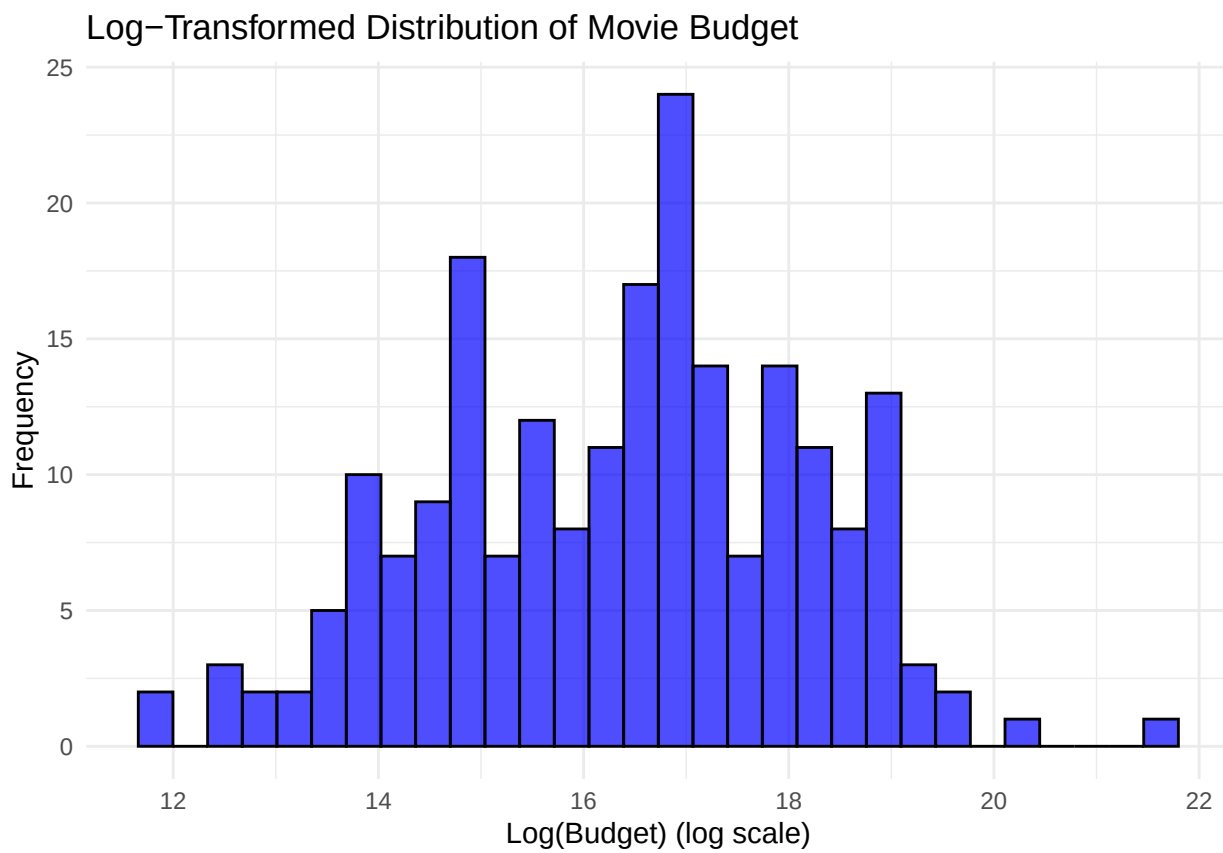
Budget

```
library(ggplot2)

ggplot(movies_clean, aes(x = budget)) +
  geom_histogram(bins = 30, fill = "blue", color = "black", alpha = 0.7) +
  scale_x_continuous(labels = scales::label_number(scale = 1e-7)) +
  labs(title = "Distribution of Movie Budget", x = "Budget (in millions)", y = "Frequency") +
  theme_minimal()
```



After observing the budget we see that there is a heavy amount of skew, likely due to the widely varying budgets between movies. As such, we will apply a log transformation to the variable and try revisualizing.

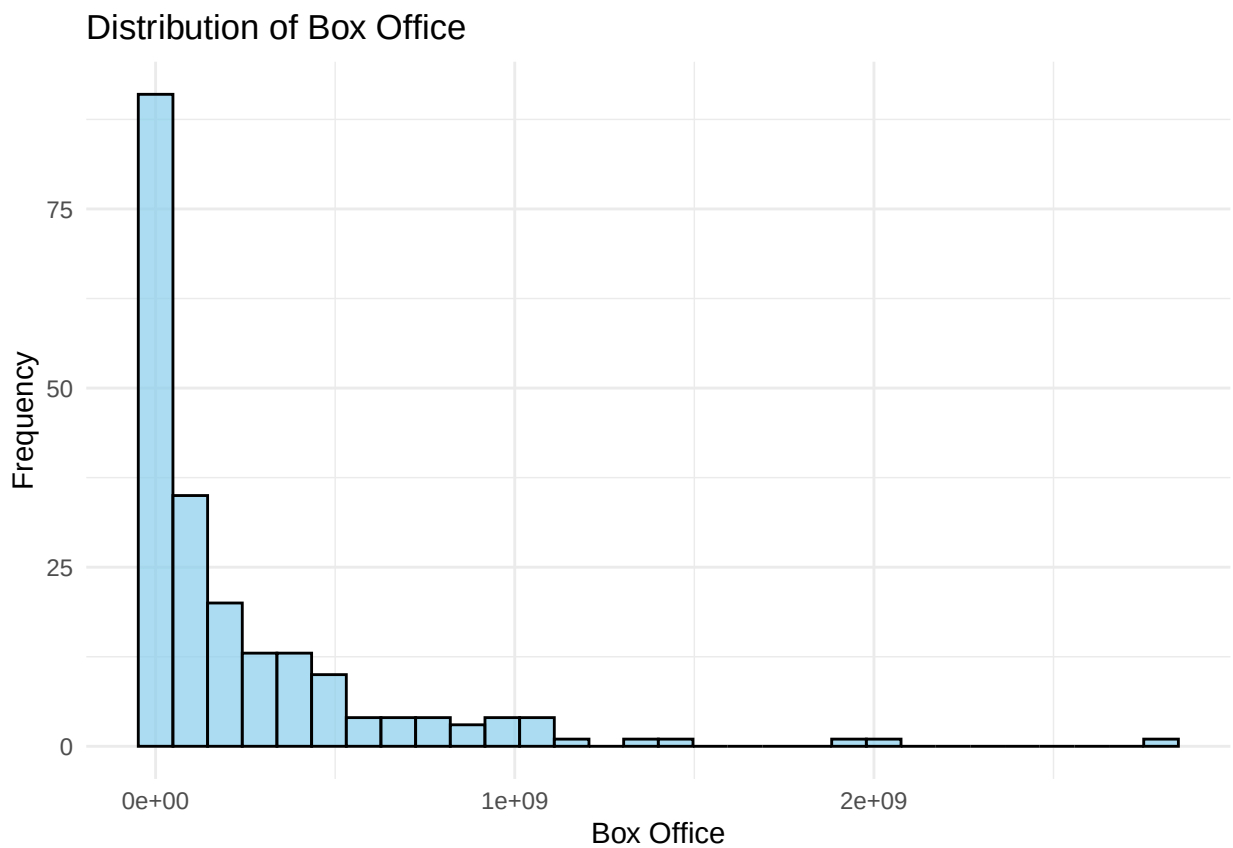
```
movies_clean$log_budget <- log(movies_clean$budget)

ggplot(movies_clean, aes(x = log_budget)) +
  geom_histogram(bins = 30, fill = "blue", color = "black", alpha = 0.7) +
  labs(title = "Log-Transformed Distribution of Movie Budget",
       x = "Log(Budget) (log scale)",
       y = "Frequency") +
  theme_minimal()
```



Now after log transformation we see a normal distribution which will allow us to build a more valid model later.

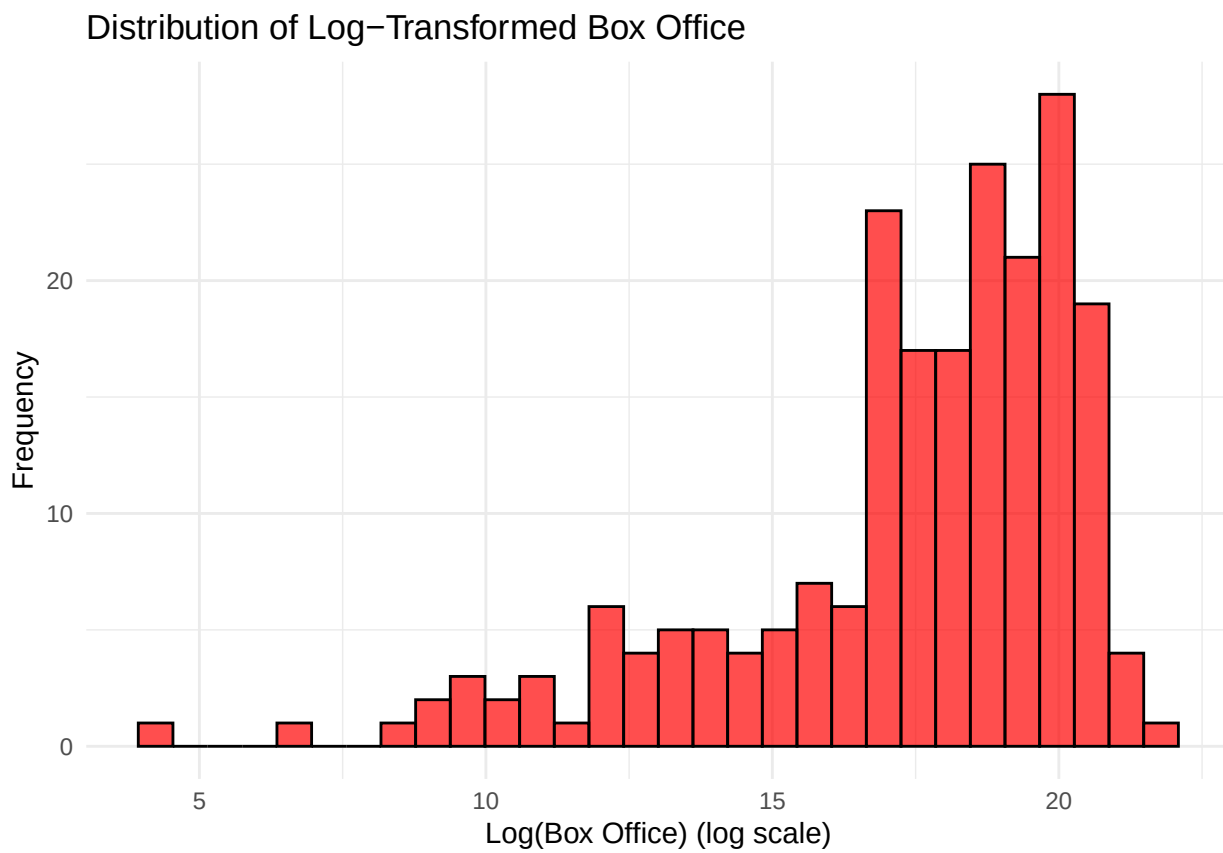
Box Office

```
ggplot(movies_clean, aes(x = box_office)) +
  geom_histogram(bins = 30, fill = "skyblue", color = "black", alpha = 0.7) +
  labs(title = "Distribution of Box Office",
       x = "Box Office",
       y = "Frequency") +
  theme_minimal()
```



Like the budget, the box office is (expectedly) very right skewed. I did not put the box office in millions for this reason as I found it to be redundant given how skewed the data is and the fact it will require a log transformation.

```
movies_clean$log_box_office <- log(movies_clean$box_office)

ggplot(movies_clean, aes(x = log_box_office)) +
  geom_histogram(bins = 30, fill = "red", color = "black", alpha = 0.7) +
  labs(title = "Distribution of Log-Transformed Box Office",
       x = "Log(Box Office) (log scale)",
       y = "Frequency") +
  theme_minimal()
```



While the log transformed box office data is still somewhat skewed, the data is a lot more normal for use in my model.

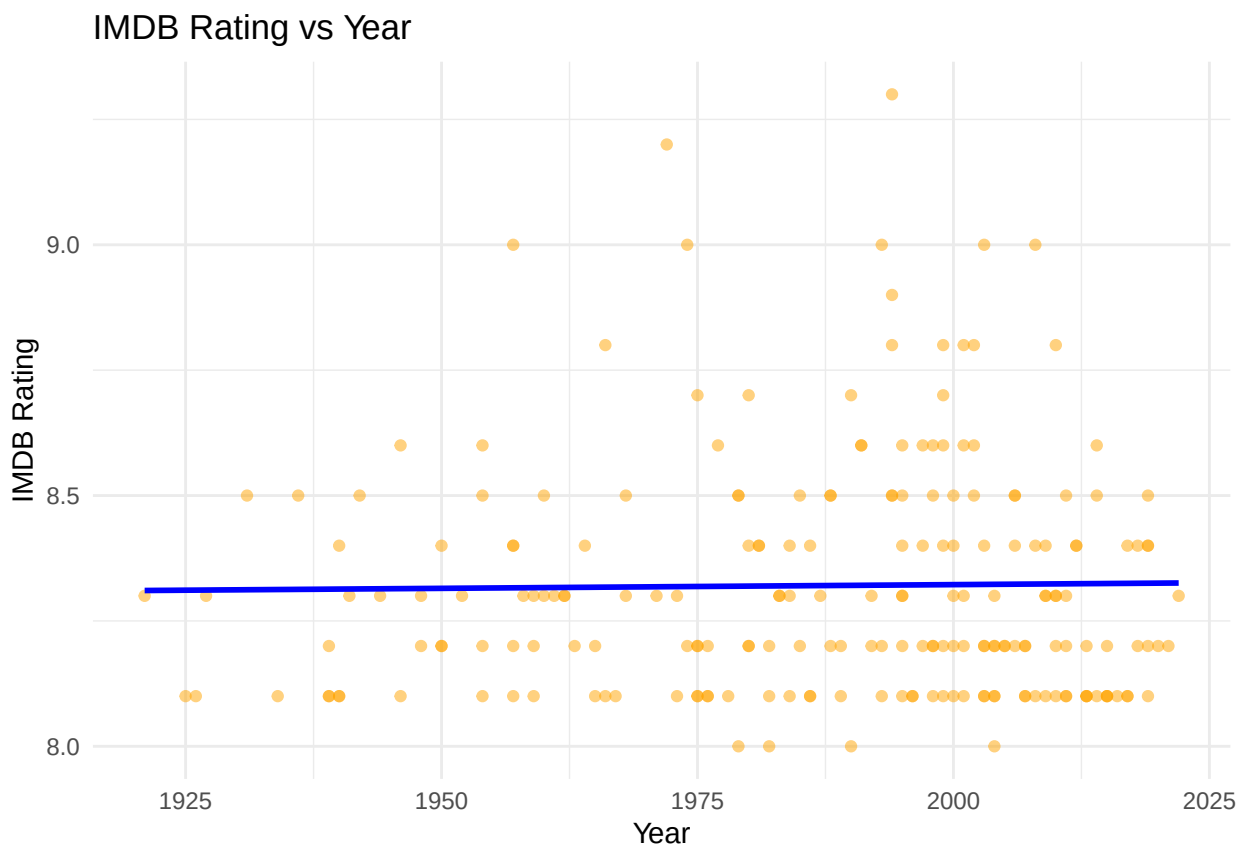
Correlational Analysis between Rating and each Variable

Now we will see if there are any standout relationships between individual variable and rating.

Rating vs Year

```
ggplot(movies_clean, aes(x = year, y = rating)) +
  geom_point(alpha = 0.5, color = "orange") +
  geom_smooth(method = "lm", color = "blue", se = FALSE) +
  labs(title = "IMDB Rating vs Year", x = "Year", y = "IMDB Rating") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

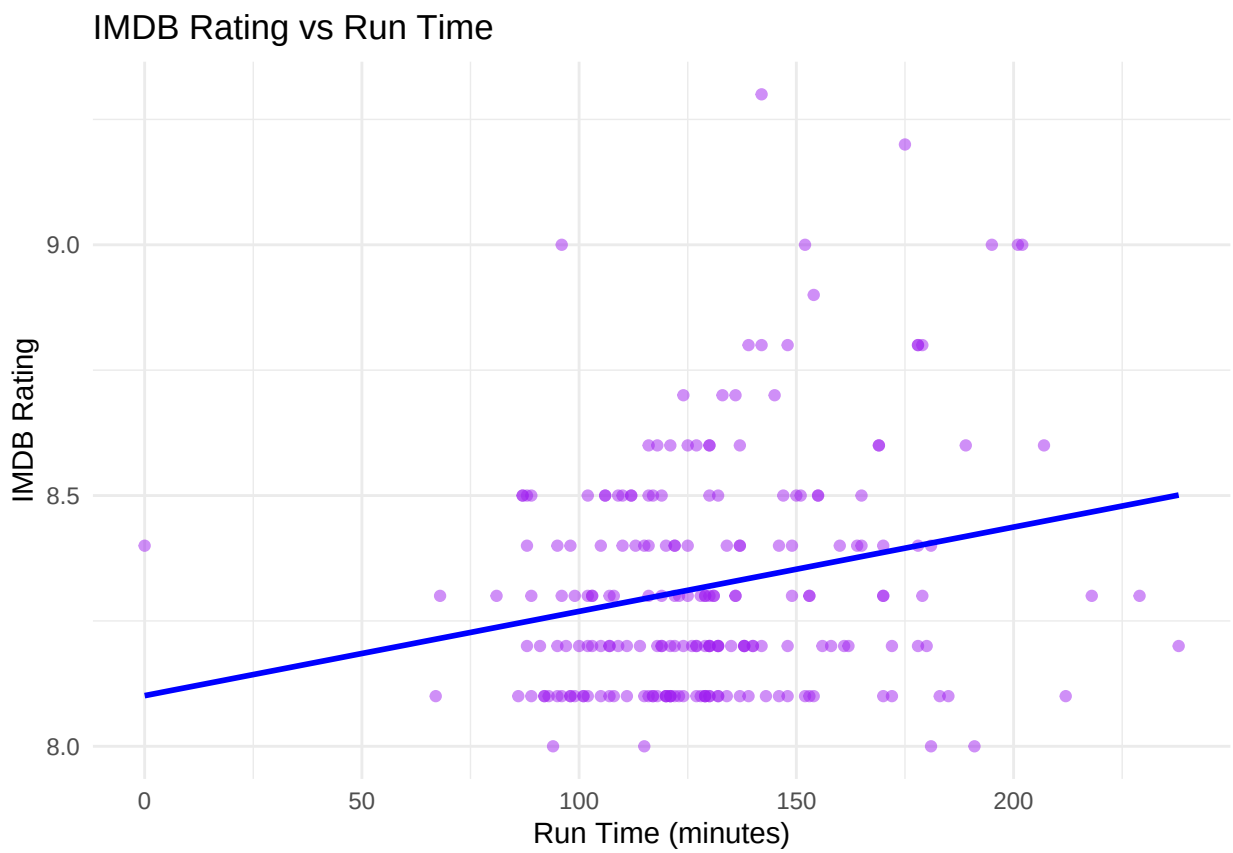


Based on this visualization, it seems that year does not much of an influence on rating at all, but I will investigate further in my model.

Rating vs Run Time

```
ggplot(movies_clean, aes(x = run_time, y = rating)) +
  geom_point(alpha = 0.5, color = "purple") +
  geom_smooth(method = "lm", color = "blue", se = FALSE) +
  labs(title = "IMDB Rating vs Run Time", x = "Run Time (minutes)", y = "IMDB Rating") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

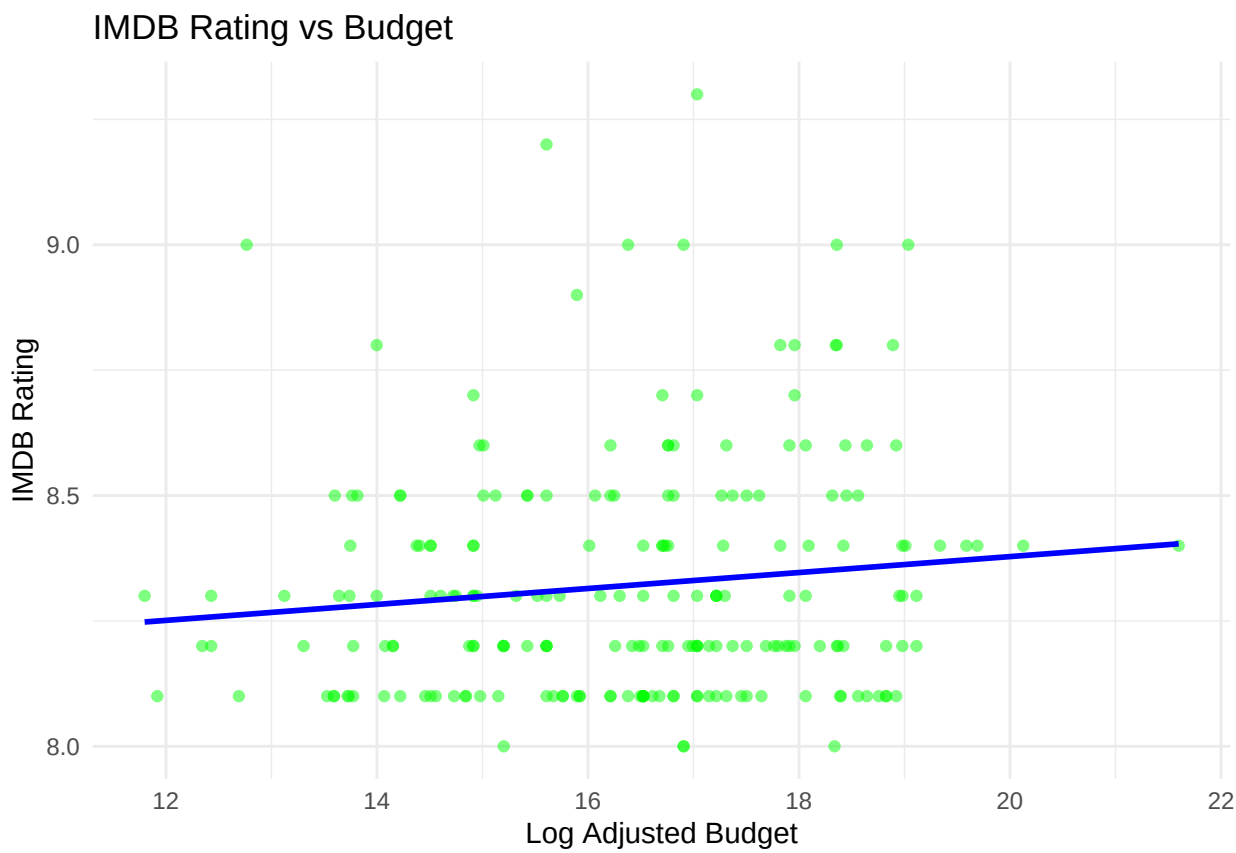


Based on this visualization, we see a slight positive correlation between rating and run time, which could indicate that run time might play a role in predicting rating.

Rating vs Budget

```
ggplot(movies_clean, aes(x = log_budget, y = rating)) +
  geom_point(alpha = 0.5, color = "green") +
  geom_smooth(method = "lm", color = "blue", se = FALSE) +
  labs(title = "IMDB Rating vs Budget", x = "Log Adjusted Budget", y = "IMDB Rating") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

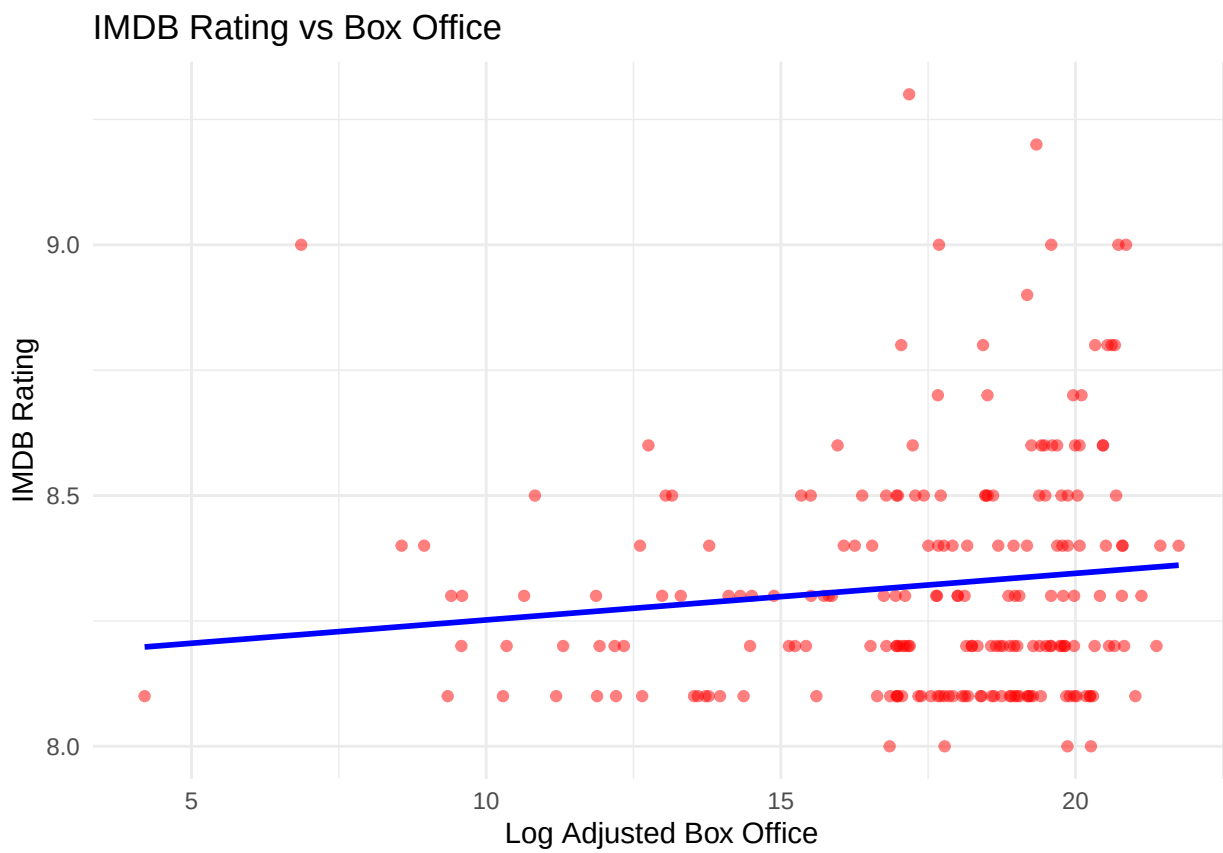


Now comparing rating with log adjusted budget, we see a slight positive correlation. This indicates that higher budget could predict higher rating, but we will investigate this further.

Rating vs Box Office

```
ggplot(movies_clean, aes(x = log_box_office, y = rating)) +
  geom_point(alpha = 0.5, color = "red") +
  geom_smooth(method = "lm", color = "blue", se = FALSE) +
  labs(title = "IMDB Rating vs Box Office", x = "Log Adjusted Box Office", y = "IMDB Rating") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



We once again see a slight positive correlation between rating and the log adjusted box office, which may indicate that higher box office may predict higher rating.

Model Building

The first rendition of my model contains all the variables (year, run_time, log_box_office, log_budget) to see their effect on rating.

```r
model1 <- lm(rating ~ year + run_time + log_box_office + log_budget, data = movies_clean)
summary(model1)
```

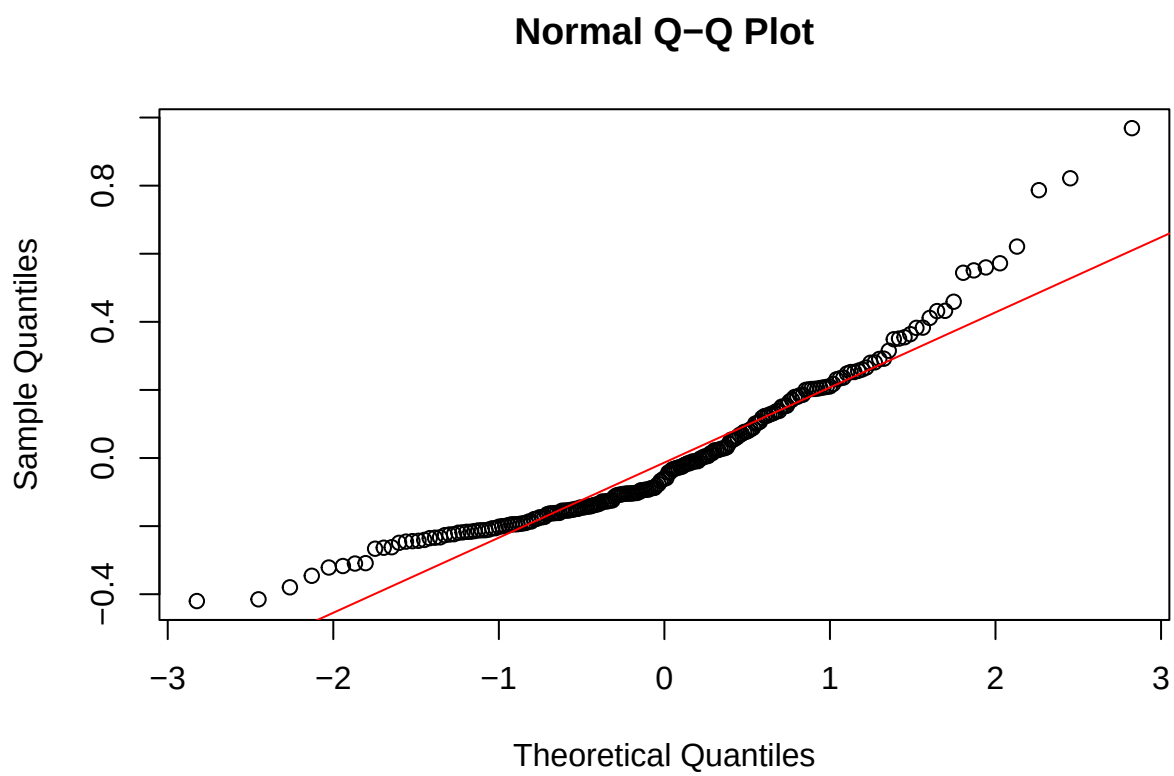
```
##
## Call:
## lm(formula = rating ~ year + run_time + log_box_office + log_budget,
##     data = movies_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.42009 -0.16214 -0.05933  0.13545  0.96883
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)    10.469469   1.916359   5.463 1.34e-07 ***
## year           -0.001335   0.001035  -1.289   0.1988
## run_time        0.001397   0.000544   2.569   0.0109 *
## log_box_office  0.009405   0.008004   1.175   0.2413
```

```
## log_budget      0.009573   0.014710   0.651   0.5159
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2325 on 206 degrees of freedom
## Multiple R-squared:  0.06306,    Adjusted R-squared:  0.04486
## F-statistic: 3.466 on 4 and 206 DF,  p-value: 0.009146
```

Based on this initial model, it appears that most of the variables are not significant predictors other than run_time, but before adjusting the model lets run through some diagnostics and see if there are specific issues with the model.

Residual normality:

```r
qqnorm(resid(model1))
qqline(resid(model1), col = "red")
```




```r
shapiro.test(resid(model1))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  resid(model1)
## W = 0.92049, p-value = 3.055e-09
```

Since we see in the Shapiro-Wilk test that the p-value is extremely small, the residuals are not normally distributed. Based on the Q-Q plot, we see that there are some notable outliers, especially at the top end; so, we will check for these outliers and see how to proceed.

```r
standardized_resid <- rstandard(model1)

movies_clean[which(abs(standardized_resid) >2), c("name", "rating")]
```

```
##                                          name rating
## 1                      The Shawshank Redemption    9.3
## 2                                 The Godfather    9.2
## 3                               The Dark Knight    9.0
## 4                         The Godfather Part II    9.0
## 5                                  12 Angry Men    9.0
## 6                              Schindler's List    9.0
## 7 The Lord of the Rings: The Return of the King    9.0
## 8                                  Pulp Fiction    8.9
```

As expected based on the Q-Q plot, it seems the top 8 highest ranked movies are all far outliers. This makes sense as top movies 7 and above all score at a 9 or above which none of the movies below it manage to do. Since these movies are quite important and notable in terms of IMDB rankings, we will NOT remove them, but it should be noted that these films have such exceptional statistics that they pull on the rest of the dataset.

Now, we will check for heteroskedasticity using the Bresuch-Pagan test.

```r
library(lmtest)
```

```
## Loading required package: zoo

##
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```r
bptest(model1)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  model1
## BP = 11.101, df = 4, p-value = 0.02545
```

Since the p-value is very small, we reject the null of constant variance and assume there is heteroskedasticity in the model. I will adjust the model using robust standard errors and check variable significance again.

```r
library(sandwich)
coeftest(model1, vcov = vcovHC(model1, type = "HC1"))
```

```
##
## t test of coefficients:
##
##                   Estimate  Std. Error t value  Pr(>|t|)
## (Intercept)    10.46946921  1.65918329  6.3100 1.679e-09 ***
## year           -0.00133467  0.00090188 -1.4799   0.14043
## run_time        0.00139723  0.00063962  2.1845   0.03006 *
## log_box_office  0.00940492  0.00927087  1.0145   0.31155
## log_budget      0.00957279  0.01220822  0.7841   0.43387
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

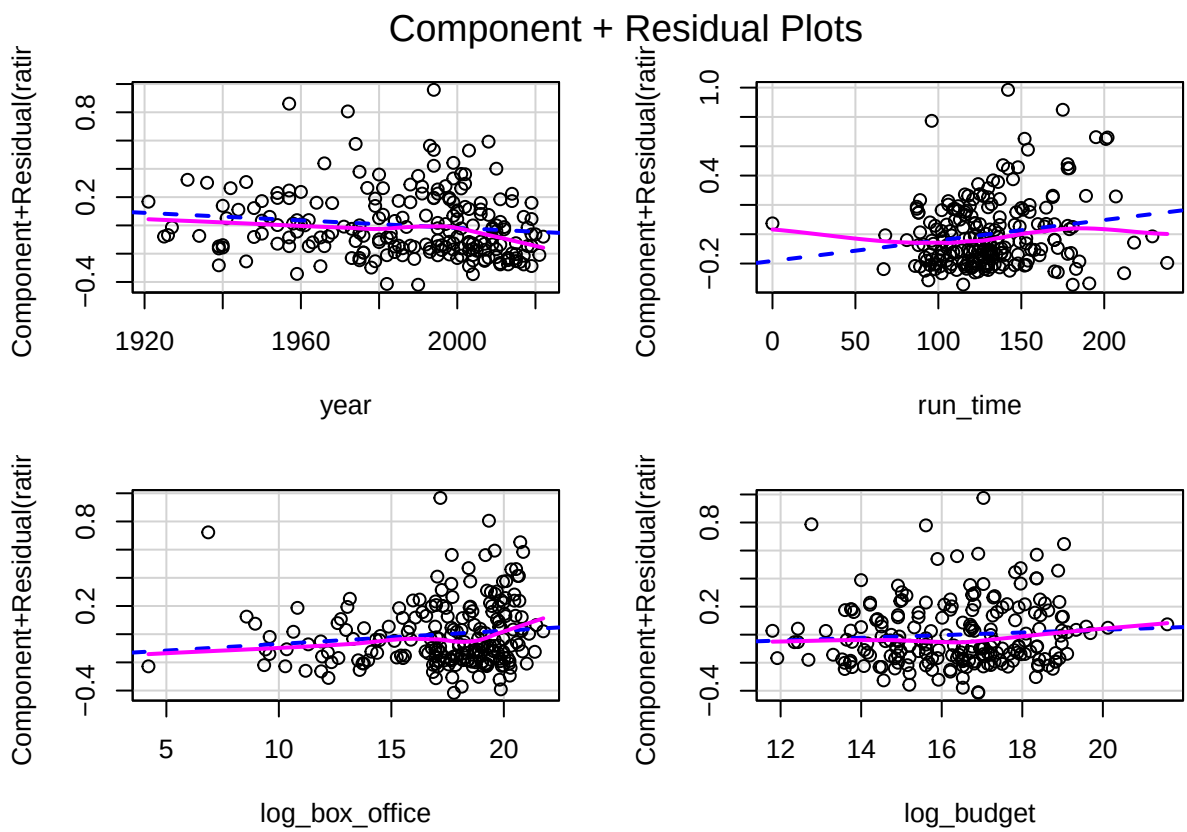
Based on the results, we see that not much has changed with only run_time still having a small enough p-value to be significant. Lets check some other diagnostics.

Correct Function Form:

```
library(car)
```

```
## Loading required package: carData

##
## Attaching package: 'car'

## The following object is masked from 'package:dplyr':
##
##     recode

## The following object is masked from 'package:purrr':
##
##     some
```

```
crPlots(model1)
```



Based on these graphs we actually find that year, run_time, and log_box_office all have some curvature to them; this could mean that we are dealing with non-linear components. To address this, lets create a second model with quadratic terms added to see if any variables change in significance.

```
model2 <- lm(rating ~ year + run_time + log_box_office + log_budget + I(year^2) + I(run_time^2) + I(log_

summary(model2)
```

```
##
```

```
## Call:
## lm(formula = rating ~ year + run_time + log_box_office + log_budget +
##     I(year^2) + I(run_time^2) + I(log_box_office^2), data = movies_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.47066 -0.15127 -0.05153  0.14284  0.99549
##
## Coefficients:
##                       Estimate Std. Error t value Pr(>|t|)
## (Intercept)         -2.025e+02  1.043e+02  -1.941   0.0536 .
## year                 2.149e-01  1.055e-01   2.036   0.0430 *
## run_time            -5.085e-04  2.495e-03  -0.204   0.8387
## log_box_office      -7.965e-02  3.879e-02  -2.053   0.0413 *
## log_budget          -2.262e-03  1.600e-02  -0.141   0.8877
## I(year^2)           -5.464e-05  2.668e-05  -2.048   0.0419 *
## I(run_time^2)        6.566e-06  8.948e-06   0.734   0.4640
## I(log_box_office^2)  3.072e-03  1.332e-03   2.306   0.0221 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2292 on 203 degrees of freedom
## Multiple R-squared:  0.1024, Adjusted R-squared:  0.0714
## F-statistic: 3.307 on 7 and 203 DF,  p-value: 0.002358
```

This updated model shows an improved adjusted R-squared from 0.0448 to 0.0714. Furthermore, the residual standard error dropped slightly from 0.232 to 0.229. We also see that run_time when accounting for the nonlinearity is actually not significant compared to when run_time was considered linear. We also find that year and log_box_office DO become significant with the quadratics added. Lets try evaluating a third model where we assume that run_time is linear whilst keeping year and log_box_office nonlinear.

```r
model3 <- lm(rating ~ year + run_time + log_box_office + log_budget +
               I(year^2) + I(log_box_office^2), data = movies_clean)

summary(model3)
```

```
##
## Call:
## lm(formula = rating ~ year + run_time + log_box_office + log_budget +
##     I(year^2) + I(log_box_office^2), data = movies_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.46318 -0.15620 -0.04798  0.14157  0.98803
##
## Coefficients:
##                       Estimate Std. Error t value Pr(>|t|)
## (Intercept)         -2.018e+02  1.042e+02  -1.937   0.0541 .
## year                 2.142e-01  1.054e-01   2.032   0.0434 *
## run_time             1.278e-03  5.427e-04   2.355   0.0195 *
## log_box_office      -7.725e-02  3.861e-02  -2.001   0.0467 *
## log_budget          -2.585e-04  1.574e-02  -0.016   0.9869
## I(year^2)           -5.452e-05  2.665e-05  -2.046   0.0421 *
## I(log_box_office^2)  2.984e-03  1.325e-03   2.252   0.0254 *
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.229 on 204 degrees of freedom
## Multiple R-squared:  0.09998,    Adjusted R-squared:  0.07351
## F-statistic: 3.777 on 6 and 204 DF,  p-value: 0.001374
```

With this third model we see that the standard error has stayed the same at 0.229 but the adjusted R-squared has improved again from 0.0714 to 0.0735. The model is improving, but still only accounts for about 7.4% of variance. After checking multicollinearity, lets see how the model should be adjusted.

Checking Multicollinearity:

```r
vif(model3)
```

```
##                year            run_time      log_box_office          log_budget
##        25925.437507            1.177241           58.029746            3.199871
##           I(year^2) I(log_box_office^2)
##        25931.850113           64.629170
```

By analyzing variance inflation factors, we see that there is some extreme multicollinearity in the year and log_box_office variables, which I suspect is due to the quadratic transformations. Lets compare with model1 which does not contain any quadratics.

```r
vif(model1)
```

```
##           year       run_time log_box_office     log_budget
##       2.425501       1.147429       2.419240       2.709734
```

We see that the initial model does not contain multicollinearity, but we are unable to capture the nonlinearity of the year and log_box_office variables. As such, despite the slight improvement in standard error and adjusted R-squared, in order to better interpret individual coefficients we will remove the quadratics.

At this point we are almost back to square one, which is somewhat disheartening, but we can now remove variables based on how we perceive them to be significant for rating.

Across all of the model renditions and diagnostics, one thing remained consistent; log_budget does not appear to be a significant predictor. Lets remove it and check the model.

```r
model4 <- lm(rating ~ year + run_time + log_box_office, data = movies_clean)
summary(model4)
```

```
##
## Call:
## lm(formula = rating ~ year + run_time + log_box_office, data = movies_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.42108 -0.16500 -0.05643  0.13245  0.97232
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)     9.9736588  1.7559411   5.680 4.52e-08 ***
## year           -0.0010312  0.0009230  -1.117  0.26519
## run_time        0.0014948  0.0005222   2.862  0.00464 **
## log_box_office  0.0115301  0.0072970   1.580  0.11561
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2322 on 207 degrees of freedom
```

```
## Multiple R-squared:  0.06113,    Adjusted R-squared:  0.04752
## F-statistic: 4.493 on 3 and 207 DF,  p-value: 0.004449
```

Compared to model1, we see very slight, almost negligible improvements to standard error and adjusted R-squared. Lets make one more adjustment by removing year. Since run_time appears to be the only significant variable at this point, I want to keep it in the model. While both year and log_box_office are considered less significant, I hypothesize that the actual year the film was created in would probably have less predictive power on the current recption of the movie compared to how well the movie performed financially at the box office.

```r
model5 <- lm(rating ~ run_time + log_box_office, data = movies_clean)
summary(model5)
```

```
##
## Call:
## lm(formula = rating ~ run_time + log_box_office, data = movies_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.41681 -0.16697 -0.06613  0.14082  0.96292
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)    8.0152300  0.1026612  78.075  < 2e-16 ***
## run_time       0.0015548  0.0005197   2.992  0.00311 **
## log_box_office 0.0058835  0.0052664   1.117  0.26521
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2323 on 208 degrees of freedom
## Multiple R-squared:  0.05547,    Adjusted R-squared:  0.04639
## F-statistic: 6.108 on 2 and 208 DF,  p-value: 0.002645
```

Now we see that the adjusted R-squared has decreased very slightly, but due to the removed variable's lack of significance it seems that removing the complexity is worthwhile. I will create one more model only including the significant variable run_time, then finally compare the relevant models' AICs (Akaike Information Criterion) to decide on the final model.

```r
model6 <- lm(rating ~ run_time, data = movies_clean)
summary(model6)
```

```
##
## Call:
## lm(formula = rating ~ run_time, data = movies_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.42197 -0.17114 -0.06401  0.13853  0.96037
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 8.1010106  0.0681844  118.81   <2e-16 ***
## run_time    0.0016804  0.0005077    3.31   0.0011 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 0.2324 on 209 degrees of freedom
## Multiple R-squared:  0.0498, Adjusted R-squared:  0.04526
## F-statistic: 10.95 on 1 and 209 DF,  p-value: 0.0011
```

Once again the adjusted R-squared is very slightly decreased, which reaffirms that run_time has been the only significant variable thus far. Before deciding on a final model, I will compare the AICs of the last few models and evaluate.

```r
AIC(model1, model4, model5, model6)
```

```
##        df        AIC
## model1  6  -9.956251
## model4  5 -11.522934
## model5  4 -12.254444
## model6  3 -12.992148
```

We see that the final model, model6, containing only run_time has the lowest AIC and is therefore the best balance of fit and simplicity. Since we have established that run_time is the most significant variable, our final model will be model 6 which contains only run_time as a predictor for IMDB rating.

Analytic Interpretations

Our final model is:

```r
summary(model6)
```

```
##
## Call:
## lm(formula = rating ~ run_time, data = movies_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.42197 -0.17114 -0.06401  0.13853  0.96037
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 8.1010106  0.0681844  118.81   <2e-16 ***
## run_time    0.0016804  0.0005077    3.31   0.0011 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2324 on 209 degrees of freedom
## Multiple R-squared:  0.0498, Adjusted R-squared:  0.04526
## F-statistic: 10.95 on 1 and 209 DF,  p-value: 0.0011
```

In the final model we have one variable of run_time which is statistically significant with a p-value of 0.001. The model can be interpreted as such: for every 1-minute increase in run time, the predicted rating increases by about 0.00168 points. Over the course of about an hour difference (60 minutes), the model predicts roughly a 0.1 points increase in rating. The standard error is 0.232 and the adjusted R-squared is 0.0452.

Discussion

Overall, this report ended up more long-winded than anticipated, but I believe it was an interesting journey exploring some variables in the IMDB Top 250 Movies dataset. In the end, I decided that run time was the variable with the most predictive power in answering the question of "What are the best predictors for IMDB ratings among the 250 movies?" In order to keep model simple and significant, I decided to only include

this variable in my final model. However, the model only accounts for 4.5% of variation in IMDB ratings amongst the top 250 movies. I heavily deliberated on adding more variables, but for a few key reasons I decided against it.

1. Run time was the only consistently significant variable.
2. Across all of the other three variables year, budget, and box office, I failed to see their predictive power both statistically and anecdotally.
3. Because the dataset is only concerned with the top 250 movies, it was difficult for me to justify complicating the model for very minor gains when the predictive model was already dealing with relatively strict parameters.

Year, budget, and box office I interpret to not be so significant in determining ratings since:

Modern critics are unlikely to evaluate year of release significantly except to provide context for certain attributes of the film.

Budget likely does not play a significant role in how critics evaluate a movie's plot, performances, and entertainment or other value.

Box office success is not reliable as a predictor because critical reception can often differ from audience perceptions, and film critics are often known to value plot, production, and actor performances over general audiences who may simply enjoy a movie if it has some relatively shallow entertainment value.

However, run time does seem to be a plausible predictor if only because a longer run time typically allows for a more extensive and in depth plot, as well as more opportunities for acting prowess to shine. I believe film critics would highly value plot and actor performance, so it makes sense that run time is a useful predictor.

An important limitation to highlight is that the dataset only contains the top 250 rated movies, so it is certainly not a holistic dataset with which to draw more general predictions from. The model I created technically only predicts movie ratings within the top 250 rated movies, not across movies as a whole. Perhaps if a larger dataset was established, variables may appear more or less significant. Another limitation is that IMDB ratings are technically not generated by industry established critics, but rather based on votes from registered users of the website. While this group of voters is likely more invested in film critique compared to audiences at large, there are a number of "x-factors" that could account for how IMDB users voted. As such, further research ought to address a larger dataset, and perhaps explore into a tangential question such as what factors contribute to audience ratings or scores from other movie rating sites such as Rotten Tomatoes.